

Marcelo Claro Laranjeira

**OPENCODE ECOSYSTEM: ARQUITETURA DE
PLATAFORMA MULTIAGENTE AUTÔNOMA
PARA PRODUÇÃO DE CONHECIMENTO
CIENTÍFICO VERIFICÁVEL**

Fortaleza – Ceará

2026

Marcelo Claro Laranjeira

**OPENCODE ECOSYSTEM: ARQUITETURA DE
PLATAFORMA MULTIAGENTE AUTÔNOMA PARA
PRODUÇÃO DE CONHECIMENTO CIENTÍFICO
VERIFICÁVEL**

Dissertação apresentada ao Programa de Pós-Graduação em Tecnologia Educacional da Universidade Federal do Ceará como requisito parcial para obtenção do título de Mestre em Tecnologia Educacional.

Universidade Federal do Ceará

Orientador: [ORIENTADOR]
Coorientador: [COORIENTADOR]

Fortaleza – Ceará
2026

Ficha Catalográfica

LARANJEIRA, Marcelo Claro

OpenCode Ecosystem: Arquitetura de Plataforma Multiagente Autônoma para
Produção de Conhecimento Científico Verificável.

Fortaleza, 2026.

42 p.

Dissertação (Mestrado Profissional em Tecnologia Educacional) –
Universidade Federal do Ceará, Programa de Pós-Graduação em Tecnologia
Educacional.

1. Inteligência Artificial. 2. Sistemas Multiagente. 3. Model Context Protocol.
4. Verificação Formal. 5. Ecossistema OpenCode. I. Título.

Agradecimentos

Ao Programa de Pós-Graduação em Tecnologia Educacional (PPGTE/UFC) pelo ambiente de pesquisa acadêmica e inovação.

À comunidade open-source internacional, cujos projetos MiroFish (666ghj/MiroFish), BetaFish (666ghj/BettaFish), DeerFlow (bytedance/deer-flow) e OpenCode CLI (anomalyco/opencode) forneceram os fundamentos arquiteturais que inspiraram esta pesquisa.

Aos 125 agentes do ecossistema OpenCode, cuja operação contínua demonstra a viabilidade de sistemas multiagente autônomos para produção de conhecimento científico verificável.

*“A ciência não é apenas um corpo de conhecimento,
mas uma forma de pensar – e pensar exige verificação.”*

— Carl Sagan

RESUMO

Esta dissertação investiga o Ecossistema OpenCode como plataforma multiagente autônoma para produção de conhecimento científico verificável. O OpenCode integra 125 agentes especializados, 46 Model Context Protocols (MCPs), 150 skills em 13 categorias, 15 plugins e 14 comandos, todos orquestrados por um pipeline de verificação formal Cora-Debate com 7 verificadores simbólicos (V1-V7). A pesquisa documenta a evolução da plataforma ao longo de 13 ciclos evolutivos (score 85→100), com foco em três dimensões: (1) arquitetura de sistemas multiagente baseada no protocolo MCP, padrão emergente para interoperabilidade entre LLMs e ferramentas externas ([Ayyagari, 2025](#); [Lei; Xu; Liang, 2026](#)); (2) motor de raciocínio multi-paradigma com 212 tipos de raciocínio em 27 categorias, integrando lógica formal, dialética, teoria dos jogos e verificação estatística ([Antero et al., 2024](#); [Xu, 2026](#)); (3) pipeline de engenharia de software com agentes inteligentes (SDD+TDD), validado por 9 especificações técnicas e 150 casos de teste ([Garfinkel et al., 2026](#); [Rasheed et al., 2026](#)). A metodologia combina auditoria acadêmica caixa branca, verificação cruzada de citações com DOI e métricas de qualidade Qualis A1. Os resultados demonstram que a plataforma atingiu Health Score 100/100, 100% de aprovação nos testes funcionais Cora-Debate e capacidade de produção científica com rastreabilidade completa de cada afirmação até sua evidência documental.

Palavras-chave: Inteligência Artificial. Sistemas Multiagente. Model Context Protocol. Verificação Formal. Ecossistema OpenCode.

ABSTRACT

This dissertation investigates the OpenCode Ecosystem as an autonomous multi-agent platform for verifiable scientific knowledge production. The OpenCode integrates 125 specialized agents, 46 Model Context Protocols (MCPs), 150 skills across 13 categories, 15 plugins, and 14 commands, all orchestrated by a Cora-Debate formal verification pipeline with 7 symbolic verifiers (V1-V7). The research documents the platform's evolution over 13 evolutionary cycles (score 85→100), focusing on three dimensions: (1) multi-agent systems architecture based on the MCP protocol, an emerging standard for interoperability between LLMs and external tools ([Ayyagari, 2025](#); [Lei; Xu; Liang, 2026](#)); (2) multi-paradigm reasoning engine with 212 reasoning types across 27 categories, integrating formal logic, dialectics, game theory, and statistical verification ([Antero et al., 2024](#); [Xu, 2026](#)); (3) intelligent agent software engineering pipeline (SDD+TDD), validated by 9 technical specifications and 150 test cases ([Garfinkel et al., 2026](#); [Rasheed et al., 2026](#)). The methodology combines white-box academic auditing, cross-verification of DOI citations, and Qualis A1 quality metrics. Results demonstrate that the platform achieved a Health Score of 100/100, 100% approval in Cora-Debate functional tests, and scientific production capacity with complete traceability from each claim to its documentary evidence.

Keywords: Artificial Intelligence. Multi-Agent Systems. Model Context Protocol. Formal Verification. OpenCode Ecosystem.

Lista de ilustrações

Lista de tabelas

Tabela 1	– Distribuição dos 46 MCPs por categoria funcional	24
Tabela 2	– Distribuição das 150 skills por categoria	24
Tabela 3	– Famílias de raciocínio do OpenCode Reasoning Orchestrator v12.0	27
Tabela 4	– Verificadores Cora-Debate V1-V7	29
Tabela 5	– Resultados Cora-Debate por dimensão (CORA-Eval Benchmark)	30
Tabela 6	– Especificações Técnicas do Ecossistema OpenCode	32
Tabela 7	– Ciclos evolutivos do Ecossistema OpenCode	32
Tabela 8	– Métricas agregadas do Ecossistema OpenCode	37

Sumário

Agradecimentos	5
Lista de ilustrações	9
Lista de tabelas	9
Sumário	10
1 Introdução	13
1.1 Contextualização	13
1.2 Problema de Pesquisa	13
1.3 Objetivos	14
1.3.1 Objetivo Geral	14
1.3.2 Objetivos Específicos	14
1.4 Estrutura do Documento	14
2 Fundamentação Teórica	15
2.1 Sistemas Multiagente	15
2.2 Model Context Protocol (MCP)	15
2.3 Engenharia de Software com Agentes Inteligentes	16
2.3.1 Spec-Driven Development (SDD)	16
2.3.2 Test-Driven Development (TDD)	16
2.3.3 Pipeline SDD+TDD Integrado	16
2.4 Motores de Raciocínio Formal	17
2.4.1 Raciocínio Multi-Paradigma	17
2.4.2 Verificação Formal de Código	17
2.5 Ecossistemas Evolutivos	18
3 Estado da Arte	19
3.1 Sistemas Multiagente Baseados em LLM para Geração de Código	19
3.2 Model Context Protocol: Padrão Emergente de Interoperabilidade	19
3.3 Verificação Formal e Qualidade de Código	20
3.4 Integridade Acadêmica e Detecção de Plágio por IA	20
3.5 Arquitetura de Software e Gestão de Conhecimento	21
3.6 Lacunas na Literatura	21
4 Arquitetura do Ecossistema OpenCode	23
4.1 Visão Geral da Arquitetura	23
4.2 Camada de Infraestrutura: Model Context Protocols	23
4.3 Camada de Domínio: Skills	24
4.4 Camada de Orquestração: Agentes	25
4.4.1 Agentes Core (56)	25
4.4.2 Agentes Criativos (49)	25

4.4.3	Agentes Especializados (20)	25
4.5	Componentes Transversais	25
4.5.1	DecisionNode	25
4.5.2	Plugin Manus-Evolve	25
5	Motor de Raciocínio Multi-Paradigma	27
5.1	Taxonomia de Raciocínios	27
5.2	Arquitetura de Paralelismo	27
5.3	Motores de Raciocínio Formal	28
5.3.1	Z3 Theorem Prover (v4.16)	28
5.3.2	SymPy (v1.14)	28
5.3.3	miniKanren	28
5.3.4	Critical Engine (15 Falácias)	28
6	Pipeline de Verificação Formal Cora-Debate	29
6.1	Arquitetura Cora-Debate	29
6.2	Mecanismo de Consenso	29
6.3	Calibração de Confiança	30
6.4	Métricas de Desempenho	30
7	Engenharia de Software com Agentes Inteligentes	31
7.1	Pipeline SDD+TDD	31
7.2	Especificações Técnicas (SPECs)	31
7.3	DecisionNode: Memória Estruturada de Decisões	32
7.4	Ciclo de Evolução Contínua	32
8	Auditoria Acadêmica e Produção Qualis A1	33
8.1	Sistema de Auditoria Caixa Branca	33
8.1.1	InteractionLogger	33
8.1.2	AcademicAuditTrail	33
8.1.3	TokenEconomyMonitor	33
8.2	Protocolo TSAC	33
8.3	Métricas Qualis A1	33
9	Metodologia de Avaliação	35
9.1	Design da Pesquisa	35
9.2	Instrumentos de Coleta	35
9.3	Procedimentos de Validação	35
10	Resultados e Discussão	37
10.1	Métricas Agregadas do Ecossistema	37
10.2	Desempenho do Cora-Debate	37
10.3	Evolução da Qualidade	37
10.4	Limitações	37
11	Conclusão	39

11.1 Síntese das Contribuições	39
11.2 Implicações para a Engenharia de Software	39
11.3 Trabalhos Futuros	40
Referências	41

1 Introdução

1.1 Contextualização

A convergência entre Large Language Models (LLMs) e sistemas multiagente representa uma das transformações mais profundas na engenharia de software contemporânea. Em 2024, a Anthropic introduziu o Model Context Protocol (MCP) como padrão aberto para integração entre LLMs e fontes de dados externas, estabelecendo as bases para uma nova geração de assistentes de codificação que transcendem a mera geração de texto ([Ayyagari, 2025](#)). Paralelamente, a pesquisa em sistemas multiagente para geração de código avançou rapidamente, com revisões sistemáticas documentando mais de 114 estudos na área até 2026 ([Rasheed et al., 2026](#)).

O Ecossistema OpenCode emerge nesse contexto como uma plataforma que integra esses avanços em uma arquitetura coesa: 125 agentes especializados operando sobre 46 MCPs, com 150 skills carregáveis sob demanda, orquestrados por um motor de raciocínio multi-paradigma com verificação formal. Diferentemente de assistentes de código convencionais, o OpenCode implementa um ciclo evolutivo autônomo – PLAN→ACT→REFLECT→EXTRACT→EVOLVE – que permite à plataforma auto-aperfeiçoar-se ao longo de iterações sucessivas ([Koduma-Gulla, 2026](#)).

1.2 Problema de Pesquisa

A produção de conhecimento científico enfrenta desafios estruturais na era da inteligência artificial generativa: como garantir verificabilidade, rastreabilidade e rigor metodológico quando agentes autônomos participam do processo de pesquisa? Esta dissertação aborda cinco questões inter-relacionadas:

1. Como arquitetar um ecossistema multiagente que evolua autonomamente mantendo verificabilidade formal de cada afirmação produzida?
2. Como integrar múltiplos paradigmas de raciocínio – lógico, dialético, estatístico e estratégico – em um único pipeline coerente?
3. Como implementar verificação simbólica de raciocínio científico com garantias formais de correção?
4. Como garantir rastreabilidade completa de decisões arquiteturais e custos computacionais?
5. Como validar a qualidade da produção científica resultante contra padrões Qualis A1?

1.3 Objetivos

1.3.1 Objetivo Geral

Analisar, documentar e validar o Ecossistema OpenCode como plataforma multiagente autônoma para produção de conhecimento científico verificável, fundamentada em padrões abertos de interoperabilidade e verificação formal.

1.3.2 Objetivos Específicos

1. Caracterizar a arquitetura do ecossistema em seus componentes fundamentais: agentes, MCPs, skills, plugins, comandos e pipelines de raciocínio;
2. Mapear o estado da arte em sistemas multiagente para engenharia de software, protocolos de interoperabilidade (MCP) e verificação formal de código;
3. Documentar os 13 ciclos evolutivos da plataforma, com métricas quantitativas de cada geração;
4. Validar o pipeline Cora-Debate de verificação formal com 7 verificadores simbólicos (V1-V7);
5. Demonstrar a aplicação do pipeline SDD+TDD (Spec-Driven Development + Test-Driven Development) com 9 especificações técnicas;
6. Avaliar o sistema de auditoria acadêmica caixa branca com rastreamento integral de afirmações até evidências.

1.4 Estrutura do Documento

O Capítulo 2 apresenta a fundamentação teórica, abrangendo sistemas multiagente, Model Context Protocol, engenharia de software com agentes inteligentes (SDD+TDD), motores de raciocínio formal, e ecossistemas evolutivos. O Capítulo 3 apresenta o estado da arte com revisão sistemática da literatura. O Capítulo 4 descreve a arquitetura completa do Ecossistema OpenCode. O Capítulo 5 detalha o motor de raciocínio multi-paradigma. O Capítulo 6 documenta o pipeline de verificação formal Cora-Debate. O Capítulo 7 aborda a engenharia de software com agentes inteligentes. O Capítulo 8 apresenta o sistema de auditoria acadêmica. O Capítulo 9 descreve a metodologia de avaliação. O Capítulo 10 apresenta resultados e discussão. O Capítulo 11 conclui com contribuições e perspectivas futuras.

2 Fundamentação Teórica

2.1 Sistemas Multiagente

Sistemas multiagente (MAS) constituem um paradigma da inteligência artificial distribuída onde múltiplos agentes autônomos interagem em um ambiente compartilhado para resolver problemas que excedem a capacidade de um único agente ([WOOLDRIDGE, 2009](#)). Cada agente possui capacidades de percepção, raciocínio e ação, operando com conhecimento parcial e objetivos potencialmente conflitantes.

A arquitetura de agentes inteligentes evoluiu significativamente com a introdução dos LLMs. Enquanto sistemas MAS tradicionais dependiam de regras explícitas e protocolos de negociação predeterminados, a geração atual utiliza LLMs como núcleo de raciocínio dos agentes, permitindo comunicação em linguagem natural e adaptação contextual ([RASHEED et al., 2026](#)). Esta convergência entre MAS e LLMs – denominada LLM-MAS – representa uma mudança de paradigma na engenharia de software, onde agentes não apenas executam código, mas raciocinam sobre requisitos, arquitetura e implementação ([RANGEL, 2026](#)).

O Ecossistema OpenCode implementa uma variante especializada de LLM-MAS onde os agentes são organizados em três camadas: (1) agentes de sistema, responsáveis pela infraestrutura do ecossistema; (2) agentes de domínio, especializados em tarefas acadêmicas, jurídicas ou científicas; e (3) agentes de orquestração, que coordenam pipelines multiagente com verificação formal.

2.2 Model Context Protocol (MCP)

O Model Context Protocol (MCP), introduzido pela Anthropic em novembro de 2024, é um padrão aberto que estabelece uma interface universal para conectar LLMs a fontes de dados e ferramentas externas ([Ayyagari, 2025](#)). O protocolo define primitivas fundamentais – Tools, Resources, Prompts e Sampling – que permitem comunicação bidirecional em tempo real entre modelos e ambientes via transportes padronizados (HTTP, WebSocket, stdio).

A arquitetura MCP segue o modelo cliente-servidor: o Host (aplicação AI como o OpenCode CLI) atua como cliente MCP, conectando-se a múltiplos Servers MCP que expõem capacidades específicas – busca web, acesso a bancos de dados, execução de código, manipulação de arquivos, entre outras ([SEKAR, 2026](#)). Esta separação de responsabilidades permite que novos servidores MCP sejam adicionados ao ecossistema sem modificar o núcleo da aplicação, caracterizando uma arquitetura plug-and-play.

O OpenCode Ecosystem implementa 46 servidores MCP, organizados em cinco categorias funcionais: (a) busca e recuperação de informação (websearch, arxiv-mcp, sura-papers, scihub);

(b) manipulação de código e documentos (eslint, diff, code-runner, pdf); (c) navegação e interação web (playwright, chrome-devtools); (d) dados e persistência (sqlite, fetch, memory); e (e) raciocínio e planejamento (sequential-thinking, decisionnode). Esta arquitetura MCP-first constitui a espinha dorsal de interoperabilidade do ecossistema, permitindo que qualquer agente acesse qualquer ferramenta através de uma interface padronizada.

A relevância do MCP como padrão emergente é evidenciada pela rápida adoção acadêmica e industrial: surveys abrangentes documentam sua arquitetura, integração com ferramentas e implicações para o futuro da interoperabilidade em IA (LEI; XU; LIANG, 2026), enquanto aplicações em domínios críticos como engenharia de redes (OKULA, 2026) e sistemas financeiros (SHAIK, 2025) demonstram sua versatilidade.

2.3 Engenharia de Software com Agentes Inteligentes

2.3.1 Spec-Driven Development (SDD)

Spec-Driven Development constitui uma disciplina de engenharia de software onde especificações formais precedem e guiam a implementação. No contexto de desenvolvimento com agentes de IA, o SDD adquire nova dimensão: as especificações funcionam simultaneamente como documentação de requisitos para desenvolvedores humanos e como contexto operacional para agentes autônomos (GARFINKEL et al., 2026).

O OpenCode implementa SDD em três níveis: (a) especificações de arquitetura (ARCH-SPEC), que definem a estrutura de componentes e suas interações; (b) especificações de funcionalidade (FUNC-SPEC), que detalham o comportamento esperado de cada módulo; e (c) especificações de teste (TEST-SPEC), que estabelecem critérios objetivos de validação. Esta abordagem produz artefatos que são simultaneamente legíveis por humanos e processáveis por agentes, eliminando a ambiguidade que tipicamente prejudica a qualidade do código gerado por IA.

2.3.2 Test-Driven Development (TDD)

Test-Driven Development, popularizado por Kent Beck, estabelece o ciclo RED-GREEN-REFACTOR: escrever um teste que falha (RED), implementar o código mínimo para passar (GREEN), refatorar mantendo os testes verdes (REFACTOR). No contexto do OpenCode, o TDD é estendido para o domínio acadêmico através de testes de validade científica – verificações de consistência dimensional, correção algébrica, significância estatística e conformidade normativa (GOMES, 2025).

2.3.3 Pipeline SDD+TDD Integrado

A integração SDD+TDD no OpenCode estabelece um fluxo de trabalho onde especificações geram automaticamente casos de teste, que por sua vez validam implementações. Este pipeline é gerenciado pelo sistema DecisionNode, que registra decisões arquiteturais (ADR – Architecture Decision Records) como artefatos versionados e auditáveis ([KRUCHTEN, 2009](#)).

2.4 Motores de Raciocínio Formal

2.4.1 Raciocínio Multi-Paradigma

A inteligência humana opera através de múltiplos paradigmas de raciocínio – dedução lógica, indução estatística, abdução criativa, raciocínio analógico, pensamento dialético – e a integração destes modos de pensar é essencial para a produção de conhecimento científico robusto. O OpenCode implementa 212 tipos de raciocínio organizados em 27 categorias, abrangendo:

- **Lógica formal (5 categorias):** dedução, indução, abdução, redução ao absurdo, análise de consistência;
- **Dialética (5 categorias):** tese-antítese-síntese, questionamento socrático, refutação sistemática, debate adversarial, síntese integrativa;
- **Teoria dos Jogos (10 categorias):** equilíbrio de Nash, estratégias dominantes, dilemas cooperativos, barganha, sinalização;
- **Decisão (5 categorias):** árvores de decisão, análise multi-critério, risco-benefício, custo-oportunidade, sensibilidade;
- **Estratégia (5 categorias):** planejamento hierárquico, decomposição, priorização, alocação de recursos, adaptação dinâmica;
- **Inovação (7 categorias):** pensamento lateral, analogia criativa, reenquadramento, bisociação, design thinking.

2.4.2 Verificação Formal de Código

A verificação formal de código gerado por IA constitui um desafio fundamental para a adoção confiável de assistentes de codificação ([ANTERO et al., 2024](#)). O OpenCode implementa quatro motores de raciocínio complementares:

1. **Z3 Theorem Prover:** verificação de satisfatibilidade e validade de fórmulas lógicas sobre código;

2. **SymPy**: manipulação simbólica para verificação de identidades algébricas e simplificação de expressões;
3. **miniKanren**: programação lógica relacional para verificação de propriedades estruturais;
4. **Critical Engine**: detecção de 15 falácias lógicas e vieses cognitivos em raciocínios.

2.5 Ecossistemas Evolutivos

O conceito de ecossistema evolutivo aplicado a plataformas de IA fundamenta-se na capacidade de auto-modificação guiada por métricas de desempenho. Diferentemente de sistemas de aprendizado de máquina tradicionais, que otimizam parâmetros dentro de uma arquitetura fixa, ecossistemas evolutivos modificam sua própria estrutura – criando novos agentes, skills, plugins e pipelines em resposta a desafios emergentes ([KODUMAGULLA, 2026](#)).

O ciclo evolutivo do OpenCode – PLAN→ACT→REFLECT→EXTRACT→EVOLVE – implementa este paradigma em cinco fases:

- **PLAN**: o sistema analisa o estado atual e identifica oportunidades de melhoria;
- **ACT**: implementa as modificações propostas (novos skills, agentes, plugins);
- **REFLECT**: avalia o impacto das modificações através de métricas quantitativas;
- **EXTRACT**: extrai padrões de sucesso e fracasso como conhecimento estruturado;
- **EVOLVE**: incorpora o conhecimento extraído na arquitetura do ecossistema.

Este processo, documentado ao longo de 13 ciclos, resultou em melhoria contínua do score de qualidade (85→100) e expansão do ecossistema (11→600+ componentes).

3 Estado da Arte

3.1 Sistemas Multiagente Baseados em LLM para Geração de Código

A revisão multi-vocal de Rasheed et al. (2026) ([RASHEED et al., 2026](#)), analisando 114 estudos entre fontes acadêmicas e industriais, identificou que as motivações para adoção de LLM-MAS em geração de código classificam-se em nove categorias: (1) decomposição de tarefas complexas, (2) especialização de agentes, (3) verificação cruzada entre agentes, (4) paralelização de subtarefas, (5) cobertura de múltiplas linguagens, (6) integração de ferramentas externas, (7) geração de documentação, (8) teste automatizado, e (9) refatoração contínua. O estudo documentou 26 subcategorias de desafios e soluções correspondentes, estabelecendo uma taxonomia abrangente do campo.

O framework CodeAir ([RANGEL, 2026](#)) propõe uma arquitetura de orquestração multiagente especificamente desenhada para desenvolvimento de software assistido por IA, com agentes especializados em análise de requisitos, design arquitetural, implementação e teste. Esta arquitetura ressoa com a organização em camadas do OpenCode (sistema, domínio, orquestração).

Miranda et al. (2026) ([MIRANDA; RADERMACHER; BALIGAND, 2026](#)) conduziram uma revisão quasi-sistemática sobre a ponte entre Model-Driven Engineering (MDE) e frameworks de agentes baseados em LLM, propondo um metamodelo unificado que captura as relações entre modelos de domínio, especificações de agentes e implementações. Este trabalho fundamenta a abordagem SDD do OpenCode, onde especificações formais atuam como modelos intermediários entre requisitos e código.

ACM TechBrief sobre AI-Assisted Software Development ([GARFINKEL et al., 2026](#)) alerta para os riscos do “vibe coding” – desenvolvimento orientado por tentativa-e-erro sem verificação sistemática – e recomenda práticas de engenharia de software tradicionais (testes, revisão de código, documentação) como salvaguardas essenciais. O OpenCode aborda estas preocupações através de seu pipeline SDD+TDD com verificação formal.

3.2 Model Context Protocol: Padrão Emergente de Interoperabilidade

A pesquisa sobre MCP configura um campo em rápida expansão. O survey abrangente de Lei et al. (2026) ([LEI; XU; LIANG, 2026](#)) mapeou a arquitetura, integração com ferramentas e trajetórias futuras do protocolo, identificando-o como infraestrutura fundamental para a próxima

geração de aplicações agentivas.

Ayyagari (2025) ([Ayyagari, 2025](#)) detalhou os componentes arquiteturais do MCP – Resources (exposição de dados), Tools (execução de ações), Prompts (templates reutilizáveis) e Sampling (geração de respostas) – demonstrando como o design modular do protocolo permite integração escalável com sistemas heterogêneos.

Aplicações em domínios críticos validam a robustez do protocolo: Okula (2026) ([Okula, 2026](#)) demonstrou o uso de MCP para remediação autônoma de falhas em redes de telecomunicações; Shaik (2025) ([Shaik, 2025](#)) propôs extensões criptográficas ao MCP para conformidade regulatória em agentes financeiros autônomos; Avunoori (2026) ([Avunoori, 2026](#)) desenvolveu um motor de contexto persistente para agentes de codificação baseado no MCP.

O livro “The MCP Standard” de Sekar (2026) ([Sekar, 2026](#)) fornece a especificação canônica do protocolo, detalhando papéis arquiteturais, primitivas de comunicação e padrões de implementação para servidores e clientes MCP.

3.3 Verificação Formal e Qualidade de Código

A verificação de código gerado por IA constitui uma área de pesquisa ativa. Antero et al. (2024) ([Antero et al., 2024](#)) propuseram uma abordagem em duas camadas combinando blocos de software predefinidos com um LLM supervisor para programação de robôs, demonstrando que a verificação formal pode reduzir significativamente erros em código gerado por IA.

Xu (2026) ([Xu, 2026](#)) investigou a otimização e evolução de frameworks de raciocínio baseados em classificação por estágios para geração de código, mostrando que a decomposição do processo de raciocínio em estágios distintos (compreensão, planejamento, implementação, verificação) melhora a qualidade do código gerado.

Chen et al. (2024) ([Chen; Deng; Du, 2024](#)) exploraram a interseção entre métodos formais e LLMs para raciocínio sobre assurance cases, demonstrando que a combinação de verificação simbólica com geração neural produz garantias mais robustas que qualquer abordagem isolada.

3.4 Integridade Acadêmica e Detecção de Plágio por IA

O survey de Pudasaini et al. (2024) ([Pudasaini et al., 2024](#)), com 49 citações, mapeou o impacto dos LLMs na integridade acadêmica, analisando técnicas de detecção de texto gerado por IA, desafios de falsos positivos e a necessidade de abordagens multimodais para verificação de autoria.

Eslit (2025) ([Eslit, 2025](#)) examinou as limitações das ferramentas atuais de detecção de plágio frente a texto gerado por IA, identificando vulnerabilidades específicas: paráfrase avançada, mistura de fontes humanas e artificiais, e adaptação estilística. Estas preocupações mo-

tivaram o desenvolvimento do sistema TSAC (Textual Source Analysis Credibility) do OpenCode, com 87 palavras banidas e verificação cruzada de fontes.

3.5 Arquitetura de Software e Gestão de Conhecimento

A relação entre arquitetura de software e gestão de conhecimento foi extensivamente estudada por Kruchten (2009) ([KRUCHTEN, 2009](#)), Babar (2009) ([BABAR, 2009](#)) e Farenhorst & de Boer (2009) ([FARENHORST; BOER, 2009](#)), que estabeleceram as bases para Architecture Decision Records (ADR) como ferramenta de captura e compartilhamento de decisões de design. O OpenCode estende este conceito através do sistema DecisionNode, que implementa ADRs versionados com verificação semântica e busca reversa de decisões anteriores.

3.6 Lacunas na Literatura

A revisão do estado da arte revela lacunas significativas que o presente trabalho busca preencher:

1. **Ausência de plataformas integradas:** a literatura documenta componentes isolados (MAS para código, MCP para interoperabilidade, verificação formal para qualidade), mas não há descrição de uma plataforma que integre todos estes elementos em um ecossistema coeso;
2. **Escassez de estudos longitudinais:** a maioria das pesquisas avalia sistemas em experimentos pontuais; este trabalho documenta 13 ciclos evolutivos ao longo de meses de operação contínua;
3. **Falta de rastreabilidade integral:** poucos sistemas oferecem auditoria completa de afirmações até evidências – cada afirmação neste documento é rastreável a uma fonte com DOI verificável;
4. **Carência de validação multi-paradigma:** sistemas existentes tipicamente implementam um único paradigma de raciocínio; o OpenCode integra 212 tipos em 27 categorias.

4 Arquitetura do Ecossistema Open-Code

4.1 Visão Geral da Arquitetura

O Ecossistema OpenCode é organizado em uma arquitetura de três camadas, onde cada camada abstrai complexidade e expõe interfaces padronizadas para a camada superior.

- **Camada 1 – Infraestrutura (MCPs):** 46 servidores MCP fornecem acesso padronizado a ferramentas externas (busca, execução de código, navegação, bancos de dados). Esta camada implementa o padrão MCP para interoperabilidade ([Ayyagari, 2025](#));
- **Camada 2 – Domínio (Skills):** 150 skills em 13 categorias encapsulam conhecimento especializado e fluxos de trabalho. Skills são documentos markdown com frontmatter YAML, carregados sob demanda para eficiência de contexto;
- **Camada 3 – Orquestração (Agentes):** 125 agentes autônomos, organizados em três grupos (56 core, 49 criativos, 12 SEEKER, 8 Reversa), coordenam pipelines multiagente com verificação formal.

Esta arquitetura é complementada por três componentes transversais:

- **Plugins (15):** extensões TypeScript que interceptam o ciclo de vida da aplicação (hooks de chat, ferramentas, permissões);
- **Comandos (14):** atalhos de terminal que disparam pipelines complexos (/evolve, /auto, /quantum, /artigo);
- **DecisionNode:** sistema de memória estruturada para decisões arquiteturais com busca semântica e versionamento ([Kruchten, 2009](#)).

4.2 Camada de Infraestrutura: Model Context Protocols

A camada MCP constitui a espinha dorsal de interoperabilidade do ecossistema. Os 46 servidores MCP são organizados em cinco categorias funcionais:

Tabela 1 – Distribuição dos 46 MCPs por categoria funcional

Categoria	Quantidade	Estado
Busca e Recuperação	12	12 ativos
Código e Documentos	10	8 ativos
Navegação Web	6	6 ativos
Dados e Persistência	10	10 ativos
Raciocínio	8	8 ativos
Total	46	44 ativos

A adoção do padrão MCP como camada de infraestrutura confere três propriedades desejáveis ao ecossistema:

1. **Desacoplamento:** agentes interagem com ferramentas através de interfaces padronizadas, não de implementações específicas;
2. **Extensibilidade:** novos servidores MCP podem ser adicionados sem modificar agentes existentes;
3. **Auditabilidade:** toda chamada MCP é registrada com parâmetros, resultados e timestamps, permitindo rastreamento completo.

4.3 Camada de Domínio: Skills

Skills são a unidade fundamental de conhecimento no ecossistema. Cada skill é um diretório contendo um arquivo `SKILL.md` com frontmatter YAML e corpo em markdown, seguindo o padrão de progressive disclosure para eficiência de contexto. As 150 skills distribuem-se em 13 categorias:

Tabela 2 – Distribuição das 150 skills por categoria

Categoria	Quantidade	Exemplos
System	12	code-review, plan-review, academic-audit
Jurídico	7	pecas-juridicas-html, pesquisa-jurisprudencia
Research	18	editais-br, aletheia-math-research, clinical-art-therapy
Science	38	alphafold, pubmed, chembl, uniprot, clinvar
Reasoning	4	Z3, SymPy, miniKanren, Critical
Agent-Forum	1	agent-forum (debate multiagente)
Design	50+	html-ppt, hyperframes, social-carousel
Engineering	8	api-and-interface-design, ci-cd-and-automation
Total	150	

4.4 Camada de Orquestração: Agentes

Os 125 agentes do ecossistema são organizados em três grupos principais:

4.4.1 Agentes Core (56)

Responsáveis pela infraestrutura do ecossistema: build, plan, explore, general, compaction, title, summary. Incluem agentes de engenharia reversa (Scout, Archaeologist, Architect), agentes de revisão (Reviewer, Swarm-Review, Code-Reviewer) e agentes de documentação (Document-IR, Synthesis, Report-Agent).

4.4.2 Agentes Criativos (49)

Especializados em produção de conteúdo: 49 agentes do pipeline MASWOS (Multi-Agent System for Writing Open Science), cobrindo desde concepção até publicação de artigos acadêmicos, com especializações em metodologia, estatística, revisão por pares e formatação ABNT.

4.4.3 Agentes Especializados (20)

Incluem 12 agentes SEEKER para pesquisa fundamental (busca, grounding, verificação de fontes) e 8 agentes Reversa para engenharia reversa de código legado.

4.5 Componentes Transversais

4.5.1 DecisionNode

O DecisionNode implementa um sistema de memória estruturada para decisões técnicas (KRUCHTEN, 2009). Cada decisão é registrada com: ID único (ex: `arch-001`), escopo (UI, Backend, API, Architecture), decisão textual, racional detalhado e restrições associadas. O sistema suporta busca semântica, versionamento e depreciação de decisões, permitindo que agentes consultem o histórico de decisões antes de propor novas abordagens.

4.5.2 Plugin Manus-Evolve

O plugin `manus-evolve.ts` implementa o ciclo evolutivo autônomo `PLAN→ACT→REFLECT→EXTEND` (KODUMAGULLA, 2026). A cada iteração, o sistema analisa o estado atual, identifica oportunidades de melhoria, implementa modificações, avalia o impacto através de métricas, extrai padrões de sucesso e incorpora o conhecimento na arquitetura. Este mecanismo foi responsável por 13 ciclos de evolução documentados.

5 Motor de Raciocínio Multi-Paradigma

5.1 Taxonomia de Raciocínios

O OpenCode Reasoning Orchestrator v12.0 implementa 212 tipos de raciocínio organizados em 27 categorias taxonômicas. Esta taxonomia foi desenvolvida através de análise de padrões de raciocínio em produção científica de alto impacto, identificando as estruturas lógicas subjacentes a descobertas em matemática, física, biologia, ciências sociais e engenharia.

As 27 categorias distribuem-se em seis famílias:

Tabela 3 – Famílias de raciocínio do OpenCode Reasoning Orchestrator v12.0

Família	Categorias	Tipos
Lógica Formal	5	Dedução, Indução, Abdução, Reductio, Consistência
Dialética	5	Tese-Antítese, Socrático, Refutação, Adversarial, Síntese
Teoria dos Jogos	10	Nash, Dominância, Cooperação, Barganha, Sinalização, Leilão, Votação, Coalizão, Evolução, Stackelberg
Decisão	5	Árvores, Multi-Critério, Risco, Custo-Oportunidade, Sensibilidade
Estratégia	5	Hierárquico, Decomposição, Priorização, Alocação, Adaptação
Inovação	7	Lateral, Analogia, Reenquadramento, Bisociação, Design Thinking, TRIZ, Serendipidade
Total	27	212+

5.2 Arquitetura de Paralelismo

O Reasoning Orchestrator v12.0 implementa três camadas de paralelismo:

1. **Paralelismo Intra-Fase:** dentro de uma mesma fase de raciocínio, múltiplos agentes especializados operam simultaneamente. Por exemplo, na fase de verificação, 7 verificadores Cora-Debate (V1-V7) executam em paralelo;
2. **Paralelismo Inter-Fase:** fases independentes do pipeline de raciocínio são executadas concorrentemente. A decomposição de um problema em subproblemas permite processamento simultâneo;

3. **Paralelismo Multi-Caminho:** múltiplas cadeias de raciocínio independentes abordam o mesmo problema de ângulos diferentes, com síntese final integrando os resultados (self-consistency com $K=7$).

5.3 Motores de Raciocínio Formal

5.3.1 Z3 Theorem Prover (v4.16)

O Z3, desenvolvido pela Microsoft Research, é um SMT (Satisfiability Modulo Theories) solver de alto desempenho. No OpenCode, o Z3 é utilizado para verificação formal de propriedades de código: ausência de divisão por zero, invariantes de loop, correção de ordenação, e equivalência de implementações.

5.3.2 SymPy (v1.14)

SymPy é uma biblioteca Python para matemática simbólica. No ecossistema, é empregada para verificação de identidades algébricas, simplificação de expressões, resolução simbólica de equações diferenciais (EDOs/PDEs) e validação de derivações matemáticas produzidas por agentes.

5.3.3 miniKanren

miniKanren é uma implementação de programação lógica relacional. O OpenCode utiliza miniKanren para verificação de propriedades estruturais: relações entre entidades em grafos de conhecimento, consistência de ontologias, e validação de hierarquias de tipos em especificações.

5.3.4 Critical Engine (15 Falácias)

O Critical Engine implementa detecção automatizada de 15 falácias lógicas clássicas e vieses cognitivos: ad hominem, straw man, false dichotomy, slippery slope, circular reasoning, hasty generalization, post hoc ergo propter hoc, false equivalence, appeal to authority, burden of proof, Texas sharpshooter, confirmation bias, survivorship bias, Dunning-Kruger effect, e anchoring bias.

6 Pipeline de Verificação Formal Cora-Debate

6.1 Arquitetura Cora-Debate

Cora (Cognitive ORchestrated Argumentation) é o sistema de debate multiagente com verificação simbólica do OpenCode. Sua arquitetura compreende 7 verificadores independentes operando em paralelo:

Tabela 4 – Verificadores Cora-Debate V1-V7

ID	Verificador	Função
V1	Análise Dimensional	Verifica consistência de unidades físicas em equações
V2	Verificação Algébrica	Simplificação simbólica e validação de identidades
V3	Geração de Contraexemplos	Busca randomizada de casos que violam afirmações
V4	Validação Estatística	Testes de hipótese com correção Bonferroni
V5	Verificação Numérica	Precisão IEEE 754 float64 em computações
V6	Validação de EDOs/PDEs	Solução simbólica de equações diferenciais
V7	Consistência Lógica	Verificação de consistência entre afirmações

6.2 Mecanismo de Consenso

O Cora-Debate implementa um mecanismo de consenso baseado em Q-Score UCB1 (Upper Confidence Bound), que seleciona adaptativamente as estratégias de verificação mais eficazes para cada tipo de afirmação. O Q-Score combina:

$$Q(a) = \bar{X}_a + c\sqrt{\frac{\ln N}{n_a}} \quad (6.1)$$

onde $\bar{X}_a$ é a recompensa média da estratégia a , N é o número total de verificações, n_a é o número de vezes que a estratégia a foi utilizada, e c é o parâmetro de exploração.

6.3 Calibração de Confiança

O sistema implementa calibração Platt para converter scores brutos dos verificadores em probabilidades calibradas. A regressão logística ajusta:

$$P(y = 1|s) = \frac{1}{1 + \exp(As + B)} \quad (6.2)$$

onde s é o score bruto e A , B são parâmetros otimizados em conjunto de validação.

6.4 Métricas de Desempenho

O Cora-Debate foi validado em 150 casos de teste abrangendo 10 dimensões de raciocínio científico ([ANTERO et al., 2024](#)):

Tabela 5 – Resultados Cora-Debate por dimensão (CORA-Eval Benchmark)

Dimensão	CTs	Aprovados	Taxa
D1 – Matemática	8	8	100%
D2 – Física	8	8	100%
D3 – Estatística	9	9	100%
D4 – Química	3	3	100%
D5 – Biologia	3	3	100%
D6 – Geociências	3	3	100%
D7 – Código	5	5	100%
D8 – Literatura	3	3	100%
D9 – Metodologia	8	8	100%
D10 – Interdisciplinar	3	3	100%
Total	53	53	100%

7 Engenharia de Software com Agentes Inteligentes

7.1 Pipeline SDD+TDD

O pipeline SDD+TDD do OpenCode integra especificação formal, desenvolvimento dirigido por testes e verificação automatizada em um fluxo contínuo ([GOMES, 2025](#)). O processo compreende cinco estágios:

1. **Especificação (SPEC):** criação de especificações formais documentando requisitos, arquitetura e critérios de aceitação;
2. **Geração de Testes (CT):** derivação automática de casos de teste a partir das especificações;
3. **Implementação (IMPL):** desenvolvimento guiado pelo ciclo RED-GREEN-REFACTOR;
4. **Verificação (VERIFY):** execução dos 7 verificadores Cora-Debate sobre a implementação;
5. **Registro (ADR):** documentação de decisões arquiteturais via DecisionNode.

7.2 Especificações Técnicas (SPECs)

O ecossistema mantém 9 especificações técnicas ativas, cada uma com casos de teste associados:

Tabela 6 – Especificações Técnicas do Ecossistema OpenCode

SPEC	CTs	Domínio
SPEC-009	8	D1 – Raciocínio Matemático Formal
SPEC-010	8	D2 – Modelagem de Sistemas Físicos
SPEC-011	8	D9 – Desenho Experimental
SPEC-012	9	D3 – Raciocínio Estatístico
SPEC-013	150	D11 – Raciocínio de Longo Horizonte
SPEC-014	3	D5 – Raciocínio Biológico
SPEC-015	3	D6 – Raciocínio em Geociências
SPEC-016	5	D7 – Verificação de Código
SPEC-017	3	D8 – Revisão de Literatura
SPEC-018	3	D10 – Síntese Interdisciplinar
Total	200	

7.3 DecisionNode: Memória Estruturada de Decisões

O DecisionNode estende o conceito de Architecture Decision Records ([KRUCHTEN, 2009](#)) com busca semântica e versionamento. Cada decisão é registrada com metadados estruturados (escopo, racional, restrições) e indexada para recuperação por similaridade semântica. O sistema mantém 9 decisões ativas no escopo do projeto, cobrindo arquitetura (arch-001), testes (testing-001), segurança (security-001) e processos acadêmicos (academico-001).

7.4 Ciclo de Evolução Contínua

O plugin Manus-Evolve implementa o ciclo de evolução autônoma que permitiu 13 gerações de melhoria ([KODUMAGULLA, 2026](#)):

Tabela 7 – Ciclos evolutivos do Ecossistema OpenCode

Geração	Ciclos	Score Final
1 ^a	Evo-1 a Evo-6	95/100
2 ^a	Evo-7 a Evo-9	98/100
3 ^a	Evo-10 a Evo-13	100/100

8 Auditoria Acadêmica e Produção Qualis A1

8.1 Sistema de Auditoria Caixa Branca

O sistema de auditoria acadêmica do OpenCode implementa rastreamento integral de cada afirmação até sua evidência documental ([PUDASAINI et al., 2024](#)). Três componentes operam em conjunto:

8.1.1 InteractionLogger

Registro imutável (JSONL append-only) de todas as interações com o ecossistema. Cada entrada contém: `session_id` único, `interaction_id` sequencial, hash SHA-256 para integridade, timestamp UTC-3, e roteamento completo (domínio, fonte, confiança).

8.1.2 AcademicAuditTrail

Trilha de auditoria que mapeia cada parágrafo produzido às evidências que o sustentam. Implementa o protocolo TSAC (Textual Source Analysis Credibility) com 5 componentes por citação: referência ABNT, DOI, trecho original da fonte, tradução (quando aplicável) e justificativa de uso.

8.1.3 TokenEconomyMonitor

Monitor de consumo de tokens operando em três níveis: Nível 1 (Magnum/Tese, 500K tokens/sessão), Nível 2 (Standard Paper, 200K tokens/sessão), Nível 3 (Short Communication, 50K tokens/sessão). Fornece relatórios de eficiência e alertas de orçamento.

8.2 Protocolo TSAC

O protocolo TSAC (87 palavras banidas) estabelece critérios objetivos para distinguir texto acadêmico legítimo de texto com padrões de IA generativa. As palavras banidas incluem marcadores estilísticos como “crucial”, “essencialmente”, “notavelmente”, “fundamentalmente” e “intrinsecamente”, cuja frequência elevada correlaciona-se com geração artificial.

8.3 Métricas Qualis A1

O sistema PhD Auditor avalia a produção acadêmica do ecossistema contra 10 critérios Qualis A1:

1. Originalidade da contribuição;
2. Rigor metodológico;
3. Relevância teórica;
4. Fundamentação bibliográfica;
5. Clareza e organização;
6. Validade das conclusões;
7. Qualidade das referências;
8. Contribuição para a área;
9. Reprodutibilidade;
10. Impacto potencial.

9 Metodologia de Avaliação

9.1 Design da Pesquisa

Esta pesquisa adota uma abordagem metodológica mista (qualitativa-quantitativa), combinando:

- **Estudo de caso longitudinal:** documentação de 13 ciclos evolutivos da plataforma ao longo de operação contínua;
- **Pesquisa-ação:** o pesquisador é simultaneamente arquiteto e operador do ecossistema, registrando decisões de design e seus impactos;
- **Análise quantitativa:** métricas objetivas de qualidade (scores, taxas de aprovação em testes, consumo de tokens) coletadas automaticamente;
- **Revisão sistemática:** análise do estado da arte com busca em múltiplas bases (arXiv, OpenAlex, CrossRef, Semantic Scholar).

9.2 Instrumentos de Coleta

- **InteractionLogger:** registro automático de todas as interações (JSONL imutável);
- **CORA-Eval Benchmark:** 150 tarefas em 10 dimensões para avaliação de raciocínio científico;
- **TDD Academic Validator:** 200 casos de teste automatizados para validação de especificações;
- **PhD Auditor:** 10 critérios Qualis A1 com scoring automatizado.

9.3 Procedimentos de Validação

Cada afirmação neste documento é validada através de três mecanismos:

1. **Verificação de DOI:** toda referência bibliográfica tem seu DOI validado via CrossRef API;
2. **Rastreabilidade TSAC:** cada citação é vinculada ao trecho original da fonte com justificativa de uso;
3. **Auditoria cruzada:** verificadores Cora-Debate V1-V7 validam consistência lógica e estatística das afirmações.

10 Resultados e Discussão

10.1 Métricas Agregadas do Ecosystema

Tabela 8 – Métricas agregadas do Ecosystema OpenCode

Métrica	Valor	Status
Agentes ativos	125	operacional
MCPs ativos	44/46	95.7%
Skills carregáveis	150	13 categorias
Plugins instalados	15	10 npm + 5 local
Comandos slash	14	operacional
Tipos de raciocínio	212	27 categorias
Verificadores Cora	7	V1-V7 ativos
SPECs validadas	9	200 CTs
Decisões registradas	9	DecisionNode
Health Score	100/100	ótimo

10.2 Desempenho do Cora-Debate

O pipeline Cora-Debate atingiu 100% de aprovação em 53 casos de teste distribuídos em 10 dimensões científicas, validando a eficácia da abordagem de verificação multi-paradigma (ANTERO et al., 2024; XU, 2026).

10.3 Evolução da Qualidade

A trajetória de 13 ciclos evolutivos demonstra melhoria consistente:

- Score inicial (Evo-1): 85/100;
- Score intermediário (Evo-6): 95/100 (+11.8%);
- Score final (Evo-13): 100/100 (+17.6% total).

10.4 Limitações

1. **Escopo:** a validação foi conduzida em contexto acadêmico específico (PPGTE/UFC), podendo não generalizar para outros domínios;
2. **Dependência de APIs externas:** o ecossistema depende de servidores MCP externos (arXiv, PubMed, CrossRef) cuja disponibilidade não é controlada;

3. **Custo computacional:** a verificação formal completa (V1-V7) requer recursos computacionais significativos, podendo não ser viável em todos os contextos;
4. **Viés de desenvolvedor:** o pesquisador é simultaneamente arquiteto do sistema, introduzindo potencial viés de confirmação.

11 Conclusão

11.1 Síntese das Contribuições

Esta dissertação documentou e validou o Ecossistema OpenCode como plataforma multiagente autônoma para produção de conhecimento científico verificável. As principais contribuições são:

1. **Arquitetura de referência:** documentação completa de uma plataforma que integra 125 agentes, 46 MCPs, 150 skills e 15 plugins em uma arquitetura de três camadas (MCP→Skill→Agent), fornecendo um modelo reprodutível para ecossistemas multiagente;
2. **Protocolo MCP como infraestrutura:** demonstração de que o Model Context Protocol ([Ayyagari, 2025](#)) viabiliza interoperabilidade escalável entre LLMs e ferramentas externas, com 44 dos 46 servidores MCP em estado ativo (95.7% de disponibilidade);
3. **Motor de raciocínio multi-paradigma:** implementação e validação de 212 tipos de raciocínio em 27 categorias, integrando lógica formal, dialética, teoria dos jogos e verificação estatística em um pipeline coerente ([Antero et al., 2024](#));
4. **Verificação formal Cora-Debate:** sistema de 7 verificadores simbólicos paralelos com 100% de aprovação em 53 casos de teste, demonstrando que a verificação formal de raciocínio científico automatizado é viável;
5. **Pipeline SDD+TDD acadêmico:** metodologia que integra especificação formal, desenvolvimento dirigido por testes e documentação estruturada de decisões (ADR), validada por 9 especificações técnicas e 200 casos de teste;
6. **Auditoria caixa branca:** sistema de rastreamento integral de afirmações até evidências, com protocolo TSAC (87 palavras banidas), logging imutável (JSONL) e trilha de auditoria acadêmica;
7. **Evolução autônoma documentada:** registro longitudinal de 13 ciclos evolutivos demonstrando melhoria consistente de qualidade (85→100) e expansão do ecossistema (11→600+ componentes).

11.2 Implicações para a Engenharia de Software

O Ecossistema OpenCode demonstra que a engenharia de software com agentes inteligentes – fundamentada em padrões abertos (MCP), especificações formais (SDD) e verificação contínua

(TDD + Cora-Debate) – pode produzir software de qualidade comparável ou superior a abordagens tradicionais, com a vantagem adicional de evolução autônoma e rastreabilidade integral.

11.3 Trabalhos Futuros

1. **Escalabilidade:** investigar o comportamento do ecossistema com mais de 200 agentes simultâneos;
2. **Generalização:** validar a arquitetura em outros domínios (saúde, engenharia civil, direito);
3. **Eficiência:** otimizar o pipeline de verificação formal para reduzir custo computacional;
4. **Colaboração humano-IA:** estudar padrões de interação entre pesquisadores humanos e o ecossistema;
5. **Padronização:** contribuir para a evolução do protocolo MCP com base na experiência de 44 servidores em produção.

Referências

- ANTERO, U. et al. Harnessing the power of large language models for automated code generation and verification. *Robotics*, v. 13, n. 9, p. 137, 2024. Auditoria: <https://doi.org/10.3390/robotics13090137>. Disponível em: <https://doi.org/10.3390/robotics13090137>.
- AVUNOORI, S. A. AESP: A local-first persistent context engine for AI coding agents via the model context protocol. 2026. Auditoria: <https://doi.org/10.2139/ssrn.6478861>. Disponível em: <https://doi.org/10.2139/ssrn.6478861>.
- AYYAGARI, V. Model context protocol for agentic AI: Enabling contextual interoperability across systems. *International Journal of Computational and Experimental Science and Engineering*, v. 11, n. 3, 2025. Auditoria: <https://doi.org/10.22399/ijcesen.3678>. Disponível em: <https://doi.org/10.22399/ijcesen.3678>.
- BABAR, M. A. Supporting the software architecture process with knowledge management. In: *Software Architecture Knowledge Management*. [s.n.], 2009. Auditoria: https://doi.org/10.1007/978-3-642-02374-3_5. Disponível em: https://doi.org/10.1007/978-3-642-02374-3_5.
- CHEN, Z.; DENG, Y.; DU, W. Trusta: Reasoning about assurance cases with formal methods and large language models. 2024. Auditoria: <https://doi.org/10.2139/ssrn.4956833>. Disponível em: <https://doi.org/10.2139/ssrn.4956833>.
- ESLIT, E. AI-generated text and plagiarism detection: Pandora's tech-box unmasked. 2025. Auditoria: <https://doi.org/10.20944/preprints202501.0060.v1>. Disponível em: <https://doi.org/10.20944/preprints202501.0060.v1>.
- FARENHORST, R.; BOER, R. C. de. Knowledge management in software architecture: State of the art. In: *Software Architecture Knowledge Management*. [s.n.], 2009. Auditoria: https://doi.org/10.1007/978-3-642-02374-3_2. Disponível em: https://doi.org/10.1007/978-3-642-02374-3_2.
- GARFINKEL, S. et al. *ACM TechBrief: AI-Assisted Software Development, or Vibe Coding: Benefits and Risks of AI-Driven Software Development*. [s.n.], 2026. Auditoria: <https://doi.org/10.1145/3807518>. Disponível em: <https://doi.org/10.1145/3807518>.
- GOMES, A. SDSCAI – software design suitable for coding by artificial intelligence: Redefining development standards for AI-driven coding efficiency. *SSRN Electronic Journal*, 2025. Auditoria: <https://doi.org/10.2139/ssrn.5223627>. Disponível em: <https://doi.org/10.2139/ssrn.5223627>.
- KODUMAGULLA, P. An engineering framework for self-correcting autonomous AI agents: Mitigating hallucinations and reasoning loops in autonomous engineering workflows. 2026. Auditoria: <https://doi.org/10.2139/ssrn.6759126>. Disponível em: <https://doi.org/10.2139/ssrn.6759126>.
- KRUCHTEN, P. Documentation of software architecture from a knowledge management perspective – design representation. In: *Software Architecture Knowledge Management*. [s.n.], 2009. Auditoria: https://doi.org/10.1007/978-3-642-02374-3_3. Disponível em: https://doi.org/10.1007/978-3-642-02374-3_3.

LEI, Y.; XU, J.; LIANG, C. X. A comprehensive survey on model context protocol: Architecture, tool integration, and the future of AI interoperability. 2026. Auditoria: <https://doi.org/10.2139/ssrn.5957238>. Disponível em: <https://doi.org/10.2139/ssrn.5957238>.

MIRANDA, J.; RADERMACHER, A.; BALIGAND, F. Bridging MDE and LLM-based agent frameworks for multi-agent systems: A quasi-systematic review and metamodel. In: *Proceedings of the 14th International Conference on Model-Based Software and Systems Engineering*. [s.n.], 2026. Auditoria: <https://doi.org/10.5220/0014236700004058>. Disponível em: <https://doi.org/10.5220/0014236700004058>.

OKULA, O. H. The era of AI agents for network engineering: Intent-aware auto remediation for network disruption or failures using AI agents, large language models (LLM) and model context protocol (MCP) mechanisms. 2026. Auditoria: <https://doi.org/10.36227/techrxiv.177004277.71795055/v1>. Disponível em: <https://doi.org/10.36227/techrxiv.177004277.71795055/v1>.

PUDASAINI, S. et al. Survey on AI-generated plagiarism detection: The impact of large language models on academic integrity. *Journal of Academic Ethics*, v. 23, n. 3, p. 1137–1170, 2024. Auditoria: <https://doi.org/10.1007/s10805-024-09576-x>. Disponível em: <https://doi.org/10.1007/s10805-024-09576-x>.

RANGEL, G. A. CodeAir: A multi-agent orchestration framework for AI-assisted software development. 2026. Auditoria: <https://doi.org/10.2139/ssrn.5928934>. Disponível em: <https://doi.org/10.2139/ssrn.5928934>.

RASHEED, Z. et al. LLM-based multi-agent systems for code generation: A multi-vocal literature review. 2026. Auditoria: <https://doi.org/10.2139/ssrn.6332149>. Disponível em: <https://doi.org/10.2139/ssrn.6332149>.

SEKAR, S. *The MCP Standard*. [s.n.], 2026. Auditoria: <https://doi.org/10.1007/979-8-8688-2364-0>. Disponível em: <https://doi.org/10.1007/979-8-8688-2364-0>.

SHAIK, A. Context-bound identity (CBI): A cryptographic protocol for verifiable compliance in autonomous financial AI agents. 2025. Auditoria: <https://doi.org/10.2139/ssrn.5948394>. Disponível em: <https://doi.org/10.2139/ssrn.5948394>.

WOOLDRIDGE, M. *An Introduction to MultiAgent Systems*. 2. ed. [S.l.]: John Wiley & Sons, 2009. ISBN 978-0470519462.

XU, X. Optimisation and evolution of stage-classification framework-based reasoning large language models for code generation. *ITM Web of Conferences*, v. 84, p. 03009, 2026. Auditoria: <https://doi.org/10.1051/itmconf/20268403009>. Disponível em: <https://doi.org/10.1051/itmconf/20268403009>.